

Worksheet — 1.3 Exchanging Data — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Answer key

Section A

1. **Key-term match** A-4, B-3, C-6, D-1, E-2, F-5, G-8, H-7

2. **RLE** - (a) (R,4) (G,2) (B,3) (W,5) — i.e. 4 R, 2 G, 3 B, 5 W. - (b) A A A B A A A C C (3 A, 1 B, 4 A, 2 C). - (c) Data with **few/no runs** of repeats — e.g. a detailed photograph or already-compressed text. Each unique value still needs a count, so the file *grows* (no repetition to exploit). RLE is lossless and only shrinks files with long runs.

3. **Symmetric vs asymmetric** - (a) **same** key; **key distribution / key exchange** problem. - (b) **public** key; **private** key. - (c) **private** key. - (d) **slower**. - (e) **symmetric** session key; then **symmetric** encryption for bulk data.

4. **Normalisation** - (a) The **Courses** column holds multiple values ("Maths, Physics") — a **repeating group** / non-atomic value. To reach 1NF, make values atomic: one course per row (or move courses to a separate table) and ensure a suitable primary key. - (b) **Transitive dependency** (TutorName depends on TutorID, a non-key attribute, not directly on the key). This breaks **3NF**. (Partial dependencies break 2NF; transitive break 3NF.) - (c) Sample 3NF design (primary keys in **bold**, foreign keys marked FK): -
Student(**StudentID** , Name, TutorID (FK)) - Tutor(**TutorID** , TutorName) - Course(**CourseID** , CourseName) - StudentCourse(**StudentID (FK)**, **CourseID (FK)**) — linking/junction table with a **composite** primary key to resolve the many-to-many between students and courses.

5. **SQL** (accept reasonable variations / casing)

```
-- (a)
SELECT Name, YearGroup
FROM Student
WHERE YearGroup = 12
ORDER BY Name ASC;

-- (b)
SELECT Student.Name, Tutor.TutorName
FROM Student
JOIN Tutor ON Student.TutorID = Tutor.TutorID;

-- (c)
UPDATE Student
SET YearGroup = 13
WHERE StudentID = 'S01';
```

6. **TCP/IP layer matching** - Application: **HTTP, SMTP, FTP** - Transport: **TCP, UDP** - Network / Internet: **IP**
- Link / Data link: **Ethernet, Wi-Fi**

7. Short answer - (a) Any two of: **source IP address**, **destination IP address**, a **sequence/packet number** (for reassembly in the right order). (Also acceptable: error-check/checksum, TTL.) - (b) **DNS translates a domain name (e.g. example.com) into an IP address** so the device can locate the server. Caching stores recently looked-up results, so repeat lookups are **faster** and reduce traffic/load on DNS servers. - (c) PageRank treats each incoming link as a "vote" of importance; a high-ranked page passes on **more of its rank** through its links, so its vote counts for more. (Damping factor ~0.85; iterates until values converge.) - (d) Client-side checks can be **viewed, bypassed or disabled** by the user (e.g. editing the page / disabling JavaScript), so they are not secure. Server-side checks are hidden and authoritative, so security-critical checks must run there.

8. Referential integrity vs ACID - (a) 1 → **Atomicity**; 2 → **Referential integrity**; 3 → **Durability**; 4 → **Isolation**; 5 → **Consistency**; 6 → **Referential integrity**. - (b) The student has actually described **referential integrity**. - Referential integrity: a foreign key value must always refer to a primary key value that exists in the linked table (no "orphan" records). - Atomicity: a **transaction** must be carried out in its entirety or not at all — if any part fails, the whole transaction is rolled back so the database never holds a half-finished result. - Key distinction: atomicity is one of the **ACID** properties of *transaction processing*; referential integrity is a *design/constraint* rule about the links between tables. They are unrelated concepts. - (c) Example: a **bank transfer** — the debit from Account A and the credit to Account B must **both** happen or **neither** happen; if the system crashes after the debit, the transaction is rolled back so money is not lost. (Accept any two-step all-or-nothing example, e.g. booking a seat *and* taking payment.)

9. ER diagrams & linking tables - (a) MEMBER–CLASS: **many-to-many (M:M)**. INSTRUCTOR–CLASS: **one-to-many (1:M)** (one instructor runs many classes; each class has one instructor). - (b) A foreign key holds **one** value per record. To link one member to many classes directly you would need many foreign key values in one field (breaking 1NF/atomicity of fields) or repeated rows duplicating member data (redundancy). So M:M cannot be represented by a single foreign key in either table — it must be decomposed into two 1:M relationships via a linking table. - (c) Booking(**MemberID (FK)**, **ClassID (FK)**) — composite primary key made of the two foreign keys. (Optionally add e.g. BookingDate; a surrogate BookingID primary key with MemberID + ClassID as FKs is also acceptable.) - (d) ER diagram:

MEMBER 1 —< M BOOKING M >— 1 CLASS M >— 1 INSTRUCTOR

- MEMBER 1:M BOOKING (one member makes many bookings)
- CLASS 1:M BOOKING (one class appears in many bookings)
- INSTRUCTOR 1:M CLASS (one instructor runs many classes)
- The original MEMBER M:M CLASS relationship has been resolved into two 1:M relationships through BOOKING.

Challenge box (model points) - Hashing is **one-way**: even if the database is stolen, attackers cannot reverse the hash back to the password (unlike encryption, which is reversible with a key). So the plaintext password is never stored. - It is **deterministic**: the same password always produces the same hash. On login, the entered password is hashed and **compared to the stored hash** — if they match, access is granted; the password itself is never compared. - A good hash function has **few collisions** (different inputs rarely produce the same hash), so a wrong password is very unlikely to produce a matching hash. (Real systems also add a *salt*.)

Section B — model answers

Q1 [2]

Full-mark response: "Lossless compression reduces the file size in such a way that the original file can be perfectly reconstructed, with no data permanently removed. Source code must be compressed losslessly because every single character is significant — if lossy compression discarded or altered any characters, the program would no longer compile or would behave incorrectly."

Mark points: - **[1]** Lossless = file size reduced with **no data permanently lost** / original can be **exactly restored**. - **[1]** Code/text must be restored exactly — any changed/missing character breaks the program (so lossy is unusable).

Q2 [3]

Full-mark response:

(i) (W,10) (B,3) (W,7)

(ii) Uncompressed: 20 pixels × 1 byte = **20 bytes**. Compressed: 3 pairs × 2 bytes = **6 bytes**. Saving = 20 – 6 = **14 bytes** (a 70% reduction).

Mark points: - **[1]** (i) Correct three pairs (W,10) (B,3) (W,7) in order (accept 10W 3B 7W notation). - **[1]** (ii) Both sizes correct: 20 bytes uncompressed **and** 6 bytes compressed (3 × 2). - **[1]** (ii) Saving = **14 bytes**.

Working check: the row is 10 W, then 3 B, then 7 W = 20 pixels ✓; 3 runs → 3 pairs → 6 bytes ✓; 20 – 6 = 14 ✓.

Q3 [2]

Full-mark response: "The student has described **referential integrity**, not atomicity. Atomicity actually means that a transaction must be executed in its entirety or not at all — if any part of the transaction fails, all of its changes are rolled back, so the database is never left with a partially completed transaction."

Mark points: - **[1]** Identifies the described concept as **referential integrity**. - **[1]** Correct definition of atomicity: transaction is **all-or-nothing** / partial transactions are rolled back.

Q4 [4]

Full-mark response:

```
SELECT Pet.Name, Owner.Surname
FROM Pet
JOIN Owner ON Pet.OwnerID = Owner.OwnerID
WHERE Pet.Species = 'Dog'
ORDER BY Owner.Surname ASC;
```

Mark points: - **[1]** SELECT with both required fields (Name, Surname). - **[1]** Correct join of the two tables on OwnerID (JOIN ... ON Pet.OwnerID = Owner.OwnerID, or equivalent FROM Pet, Owner WHERE Pet.OwnerID = Owner.OwnerID). - **[1]** WHERE Species = 'Dog' (string quoted). - **[1]** ORDER BY Surname ASC (ASC may be omitted as it is the default).

(Accept aliases and reasonable dialect variations; all four clauses must combine into one valid statement.)

Q5 [3]

Full-mark response: "Isolation, one of the ACID properties, guarantees that the two simultaneous transactions cannot interfere with each other — the final result must be the same as if they had executed one after the other. The DBMS enforces this with record locking: when the first customer's transaction begins updating the seat record, that record is locked, so the second customer's transaction cannot read or write it. The second transaction must wait until the first commits and releases the lock; by then the seat is marked as sold, so the second booking is refused rather than duplicated."

Mark points: - **[1]** Isolation: concurrent transactions must not interfere / result equivalent to running them one after another (serially). - **[1]** Record locking: the seat's record is locked while the first transaction is in progress, preventing the second from accessing/updating it. - **[1]** The second transaction waits (or is rejected) until the lock is released; it then sees the seat as already booked, so it cannot be sold twice.

Q6 [3]

Full-mark response: "The browser sends the domain name to its local DNS resolver (usually at the ISP). Because the name is not cached anywhere locally, the resolver queries the DNS hierarchy: a root name server directs it to the `.com` top-level-domain server, which directs it to the authoritative name server for `example.com`. The authoritative server returns the matching IP address, which is passed back to the browser (and cached for future use); the browser can then request the page from that IP address."

Mark points: - **[1]** Request sent to a DNS resolver/server (e.g. ISP's) to look up the name. - **[1]** Query passed up/along the hierarchy: root server → TLD (`.com`) server → authoritative name server for the domain. - **[1]** IP address returned to the client (and cached), allowing the browser to contact the web server.

Q7 [4]

Full-mark response: "When Ben's browser connects, the server sends its public key (inside its digital certificate). The browser generates a random symmetric session key and encrypts it using the server's public key; only the server, holding the matching private key, can decrypt it — so the session key is exchanged securely even over an insecure network, solving the key-distribution problem. From then on, Ben's card details are encrypted with that symmetric session key. Both are needed because asymmetric encryption is too slow for bulk data but solves key exchange, while symmetric encryption is fast but needs a safe way to share its key."

Mark points: - **[1]** Server provides its **public key** (via its certificate) to the browser. - **[1]** Browser creates a **symmetric session key** and encrypts it with the server's public key; only the server's **private key** can decrypt it (secure key exchange). - **[1]** The card details / bulk data are then encrypted using the **symmetric** session key. - **[1]** Why both: asymmetric solves the **key distribution problem** but is **slow**; symmetric is **fast** for bulk data — so asymmetric is used only to exchange the symmetric key.

Q8* [9] — levels of response

Indicative content:

Benefits of the normalised relational database: - Each fact (owner's address, phone) stored **once** → reduced **data redundancy**, less storage, less re-typing. - **Consistency/no update anomalies:** changing an owner's phone number in one place updates it everywhere; the flat file risks some rows being updated and others not. - No **insertion/deletion anomalies** (e.g. deleting a pet's last appointment row in the flat file could delete the only copy of the owner's details). - **Data integrity** enforced by the DBMS: data types, validation, **referential integrity** (an appointment cannot reference a pet that doesn't exist). - Powerful

SQL querying and reporting (e.g. all unpaid invoices per owner); **concurrent multi-user access** with locking; access rights/security per table; transaction processing (ACID) for payments.

Drawbacks / practical impact: - **Cost and time**: DBMS software/hosting, designing the normalised schema, migrating and cleaning existing spreadsheet data. - **Training**: staff know the spreadsheet; SQL/database front-ends require training; a custom interface may need to be built. - **Complexity**: normalised data is split across tables, so simple "see everything in one grid" viewing now needs joins; queries across many tables can be slower to write and (for some reports) to run. - Risk during migration (data loss/errors) and dependence on technical support.

Recommendation (either way if justified — the strong answer usually recommends migrating): the long-term integrity, consistency and multi-user benefits outweigh one-off migration costs for a *growing* practice handling payments.

Levels: - **Level 3 (7–9 marks)**: A thorough discussion of benefits **and** drawbacks, explicitly using technical concepts (redundancy, update/insertion/deletion anomalies, referential integrity, concurrency/transactions) **applied to the vet practice scenario**. There is a reasoned conclusion/recommendation that follows from the argument. Well-structured line of reasoning. - **Level 2 (4–6 marks)**: Some discussion of both sides with mostly accurate use of terminology, but points may be underdeveloped or only partly applied to the scenario; conclusion present but weakly justified, or one side clearly dominates. - **Level 1 (1–3 marks)**: A few basic, generic points (e.g. "less repeated data", "costs money") with limited technical vocabulary and little or no application to the scenario; no meaningful conclusion. - **0 marks**: No creditworthy response.

Model Level-3 answer:

"In the current flat file, the same owner's name, address and phone number are duplicated on every row for each pet and appointment. This redundancy wastes storage and staff time, but more importantly it threatens data integrity: if an owner changes phone number, every one of their rows must be found and edited, and missing one creates inconsistent data (an update anomaly). Deleting a pet's only appointment row could also delete the sole record of the owner's contact details (a deletion anomaly).

A normalised relational database eliminates these problems. Owners, pets, appointments and payments would each become separate tables linked by primary and foreign keys, so each fact is stored exactly once. The DBMS then enforces referential integrity — an appointment cannot be created for a pet that does not exist, and an owner with registered pets cannot be accidentally deleted. Because payments are involved, the DBMS's transaction processing (ACID properties, e.g. atomicity so a payment is never half-recorded) and record locking allow several receptionists to use the system at once safely, which a shared spreadsheet cannot do reliably. SQL would let the manager produce reports, such as all owners with unpaid bills, in seconds.

However, migration has real costs: the practice must buy or host a DBMS, design the normalised schema, cleanse and import the spreadsheet data, and train staff who are comfortable with the spreadsheet; a user-friendly front-end will probably need to be built, and there is a short-term risk of errors during migration. Data spread across several tables is also less immediately 'visible' than one big grid.

Overall, because the practice is growing, handles payments and needs several staff to access accurate data simultaneously, the integrity, consistency and concurrency benefits clearly outweigh the one-off migration and training costs. I recommend moving to the normalised relational database, migrating in stages and running the spreadsheet in parallel until the new system is verified."